

HOME NETWORK SECURITY LAB

Project Portfolio

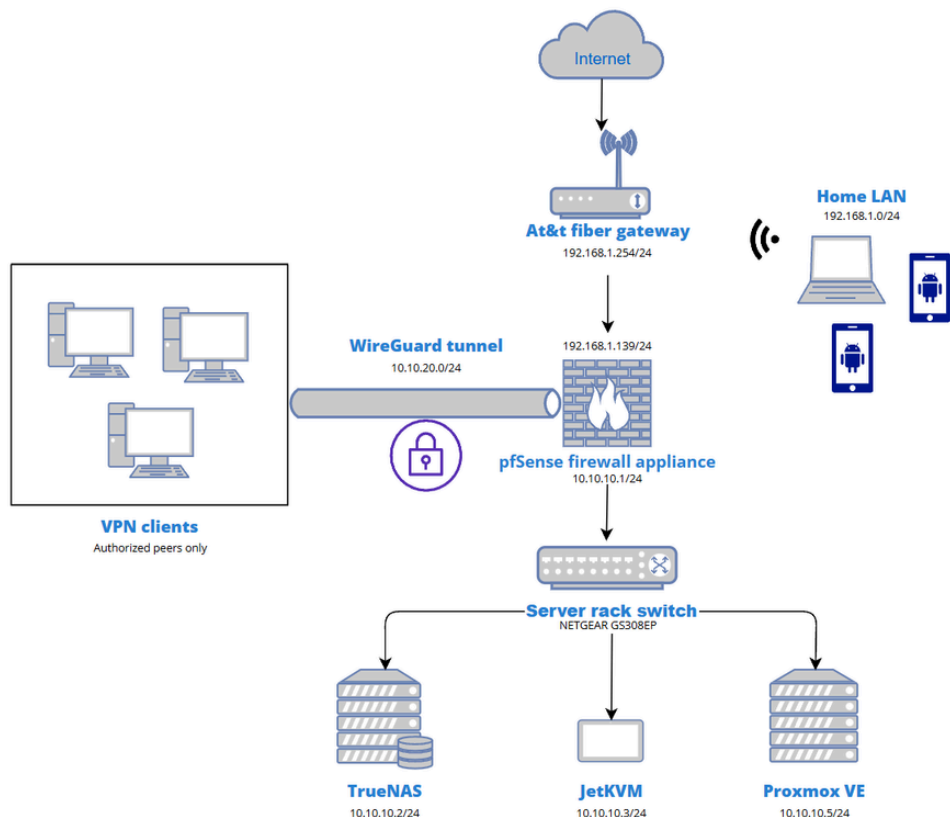
ChristianAlverio | christianjalverio@gmail.com | CoralSprings,FL

OVERVIEW

This project is a fully self-built home network and server infrastructure designed and administered from the ground up, with an emphasis on network security, access control, and monitoring. What began as a personal NAS setup grew into a segmented, multi-layer environment covering firewall administration, VPN-based remote access, intrusion detection, PKI/certificate management, virtualization, and self-hosted application deployment. The project was built incrementally over several months, with each addition treated as a real design decision — not just what is required to make it work, but what configuration would be most secure and most observable in production.

ENVIRONMENT

- Physical topology: ISP fiber gateway to pfSense firewall/router to segmented internal LAN, running behind a double-NAT boundary.
- Core infrastructure: pfSense CE (firewall/router/VPN/IDS), TrueNAS SCALE (network storage), Proxmox VE (virtualization host) — TrueNAS and Proxmox each run on their own dedicated mini PC.
- Remote access: self-hosted WireGuard VPN, no reliance on third-party mesh VPN services. Access is scoped per peer at the firewall — full LAN access for trusted devices, single-host access for everyone else, as shown in the topology diagram below.
- Virtualized/containerized services: Docker (via LXC on Proxmox), managed through Portainer.



My Network Topology

PARTS USED

- Rack — GeekPi 8U, 10-inch mini network rack.
- Firewall/router node — Fanless mini PC firewall appliance (multi-port LAN), running pfSense.
- Switch — NETGEAR 8-Port PoE+ Gigabit Smart Managed Switch (GS308EP).
- Compute nodes (x2) — Small-form-factor mini PCs — one dedicated to Proxmox VE, one dedicated to TrueNAS SCALE.
- NAS storage — Seagate IronWolf 4TB 5400 RPM SATA III 3.5" NAS hard drive, in a 3.5" hot-swap-style caddy.
- Storage expansion — PCIe-to-SATA expansion card, PCIe riser card, and SATA 22-pin extension/adaptor cabling to connect the 3.5" drive to the compute node.
- Power supply — Apevia ITX-PFC500W Mini-ITX/Flex 500W power supply, powering the drive and rack electronics.
- Power distribution — Rack-mounted PDU (125V/15A/1875W, surge-protected).
- Battery backup — CyberPower UPS, monitored through TrueNAS's NUT service.
- Networking cabling — Cat6 snagless patch cables (10Gbps-rated).
- Remote/out-of-band management — JetKVM (open-source KVM-over-IP).
- Cooling — USB-powered 120mm case fan.
- Cable testing — Klein Tools LAN cable tester.
- Custom mounts — Self-designed and 3D-printed mounting brackets, sized to fit the rack's internal layout.

PHYSICAL BUILD

The infrastructure runs on a compact GeekPi 10-inch rack, built and wired from individual components — including the NETGEAR switch, HDD caddy, JetKVM for out-of-band access, and custom 3D-printed mounting brackets designed to fit the rack's dimensions. TrueNAS and Proxmox each run on their own dedicated mini PC, with the NAS drive's power and data cabling routed through a cutout added to the TrueNAS unit's case.



Back view of the rack in progress.



Front view of the rack in progress.



3D printing a custom mounting bracket for the rack.



3D-printed bracket test-fit with a mini PC before mounting.



Power supply spec — Apevia 500W ITX PSU. The power that spins the HDD and 1875W PDU for Serverrack power.



Seagate IronWolf 4TB NAS drive in its mounting caddy, ready for installation.



Build in progress — one mini PC opened up with a PCIe-to-SATA expansion card with SATA data cabling routed through a cutout in the case.



The two “brains” of the server: TrueNAS (left) and Proxmox (right).



Structured patch cabling into the NETGEAR switch, with the PDU below, HDD caddy and JetKVM installed.



Completed ServerRack mounted on the wall shelf.



Power and data cabling routed to the drives inside the rack.



The power supply mounted in the rack, feeding power to the drives and compute nodes.



Cable routing inside the rack (i tried)



Physical network layout. PfSense box (bottom left)Segmenting Home LAN and securely connecting WAN.



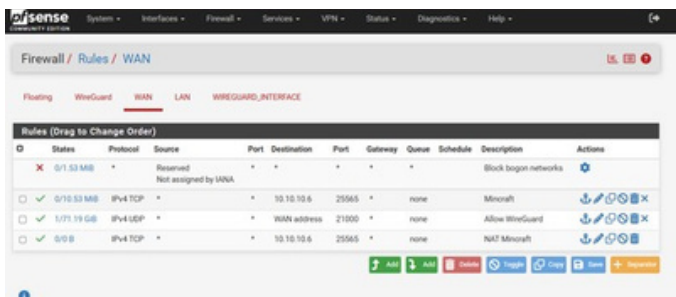
Closer view of PfSense box connecting WAN(At&t/Home LAN) and LAN(ServerRack).



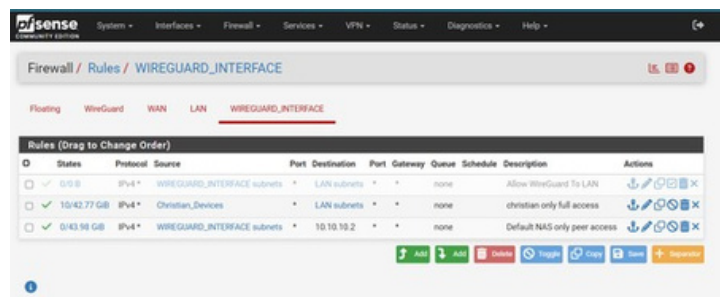
The CyberPower UPS providing battery backup, monitored via NUT.

NETWORK SECURITY & FIREWALL (PFSENSE)

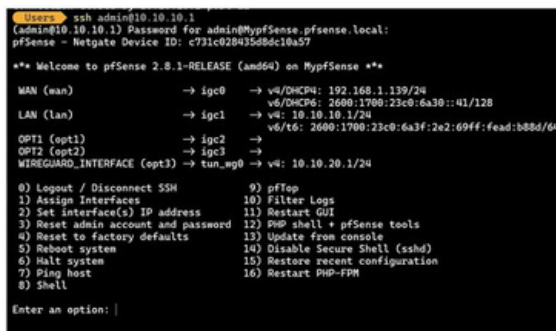
Configured and administer a pfSense firewall as the network's perimeter defense, managed through both the web GUI and direct SSH access. On the WAN side, firewall rules follow a least-privilege, default-deny model — inbound traffic is blocked by default except for two explicitly scoped allow rules (the WireGuard VPN and the Minecraft server), each locked to a single destination host and port. The LAN side is trusted, physically-controlled hardware and is intentionally left open internally, since access control for less-trusted traffic is enforced separately at the WireGuard interface rather than at the LAN rule set. Set up DHCP (Kea), DNS resolution, NTP, and system logging, and configured WAN-facing settings deliberately, including reasoning through bogon vs. private-network filtering trade-offs when they conflicted with legitimate internal testing traffic.



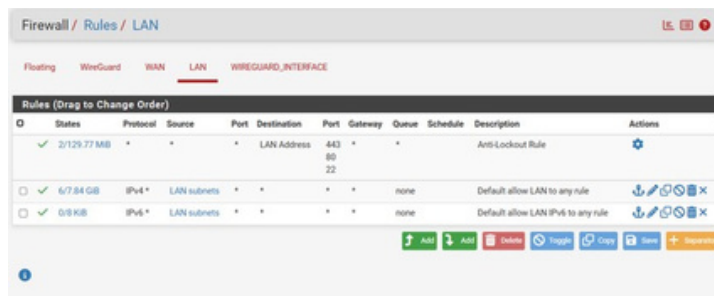
The pfSense WAN firewall rules — only Minecraft and WireGuard traffic are let in, everything else is blocked by default.



WireGuard firewall rules — full access for Me, NAS-only access for other peers.



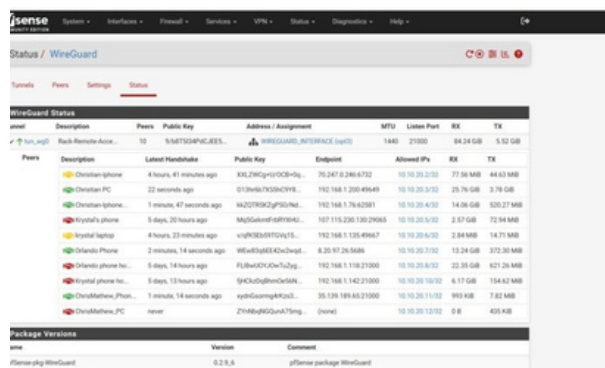
SSH access into pfSense for command-line administration.



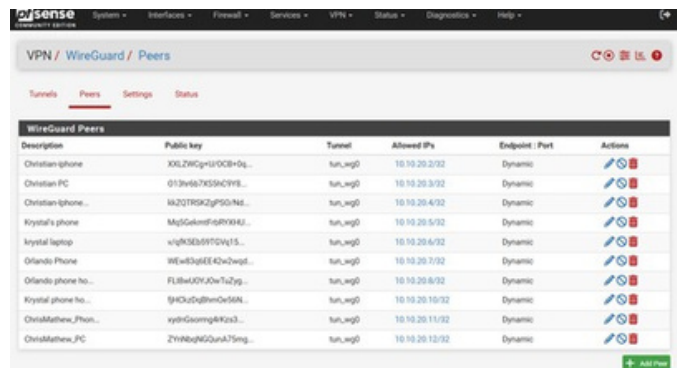
PfSense LAN (ServerRackDevices) Allow all.

VPN INFRASTRUCTURE (WIREGUARD)

Replaced a third-party mesh VPN with a fully self-hosted WireGuard deployment on pfSense, including port forwarding through a double-NAT ISP gateway. Designed a tiered access model using firewall aliases: my own devices receive full internal network access, while every other peer — family and friends I've onboarded so far — is restricted by default to a single internal host, enforced entirely at the firewall layer rather than by trusting client configuration. Generated and distributed per-peer keys and preshared keys for each device, and diagnosed a real-world NAT hairpin/reflection issue that was silently dropping same-network test traffic, tracing it back to a WAN interface bogon-filtering setting.



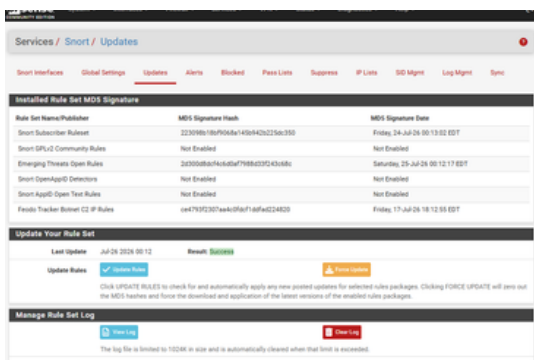
WireGuard latest handshake status board



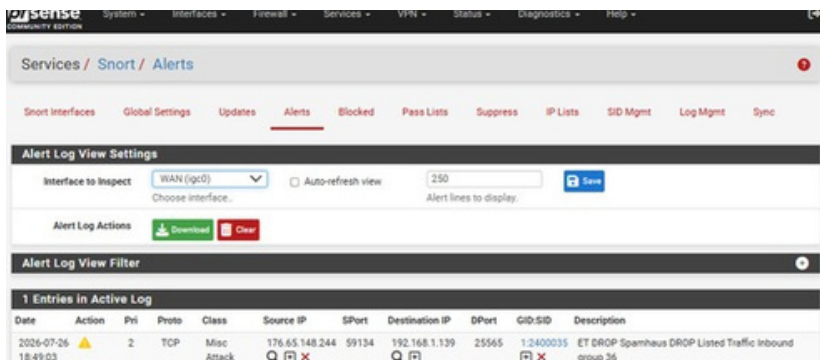
WireGuard peer configuration.

INTRUSION DETECTION & PREVENTION (SNORT)

Deployed Snort as an active IDS/IPS on the WAN interface, in inline blocking mode rather than passive logging-only mode. Curated rule categories (Snort VRT/Subscriber, Emerging Threats Open, FEODO Tracker, GPLv2 Community, OpenAppID) balanced against the hardware's processing budget, and corrected a self-lockout risk in the block-target configuration before it could firewall off the router's own WAN address. The alert log shows Snort actively catching real inbound traffic, including a hit from Spamhaus's DROP list — a feed of networks known for hosting spam and malicious activity — confirming the rule set is working against live traffic, not just configured and idle. Built a lightweight shell script, scheduled via cron, that tails Snort's alert log and pushes new detections to a Telegram bot in real time, giving immediate visibility into blocked and flagged traffic instead of relying on periodically checking a dashboard.



Snort rule installed categories



Snort Alerts

Built and manage a Proxmox VE host running a containerized services environment (Docker inside an LXC container, managed with Portainer and verified directly via the container CLI). Deployed and secured multiple self-hosted applications: Vaultwarden (password manager) behind a Caddy reverse proxy terminating TLS with an internally issued CA certificate — installed and trusted on client devices, and confirmed secure in the browser — and Uptime Kuma for service and endpoint monitoring with Telegram alerting. Scheduled automated Proxmox backups of the LXC container to TrueNAS over NFS, running on both a daily and weekly schedule with retention limits, and confirmed successful snapshot storage on the NAS side. Diagnosed and resolved a chain of real infrastructure issues along the way, including Docker's default directory permissions blocking a file share, NAT/port-forward misconfigurations, and container resource sizing.

PROXMOX Virtual Environment 9.2-4

Server View

Dashboard

proxmox

100 (KXC-Docker)

Network (proxmox)

Network Interface (proxmox)

Local (proxmox)

Container 100 (KXC-Docker) on node 'proxmox'

No Tags

Start

Shutdown

Console

More

Summary

root@KXC-Docker:~# docker ps

CONTAINER ID

IMAGE

COMMAND

CREATED

STATUS

PORTS

78c86cd989ff	itop/minecraft-server:latest	"/itop/scripts/start"	25 hours ago	Up 7 minutes (healthy)	0.0.0.0:25565->25565/tcp, [::]:25565->25565/tcp
a4a3bc38866	louislan/uptime-kuma:2	"/usr/bin/dumb-init ..." /usr/bin/kuma-init ..."	5 days ago	Up 7 minutes (healthy)	0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
5545ef94d0ff	caddy:latest	uptime-kuma "caddy run --config ..." caddy	7 days ago	Up 7 minutes	80/tcp, 2015/tcp, 443/udp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
4dcf3c9c293	vaultwarden/server:latest	"/start.sh"	7 days ago	Up 7 minutes (healthy)	80/tcp
0d3847653688	portainer/portainer-ce:latest	"/portainer"	7 days ago	Up 7 minutes	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9000/tcp
root@KXC-Docker:~#					

Resources

Network

DNS

Options

Task History

Backup

Replication

Snapshots

Firewall

Permissions

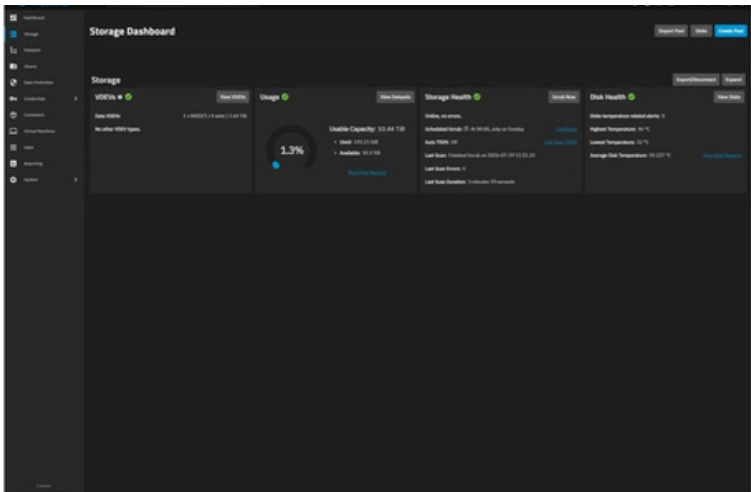
[illegible]

The screenshot displays the Proxmox VE 8.2-4 interface. The left sidebar shows the navigation menu with 'Host Overview' selected. The main panel shows the 'Host Overview' for a VM named 'jessica'. The top section displays system statistics: CPU usage (0.40% of 12 CPUs), Memory usage (0.05% of 16.0 GB), and Disk I/O (0.05% of 8.00 GB). Below this, there are two line graphs: 'CPU Usage' and 'Server Load'. The 'CPU Usage' graph shows a peak around 0.8, and the 'Server Load' graph shows a peak around 0.8. The bottom of the page shows the 'Tools' section with a 'Download' button.

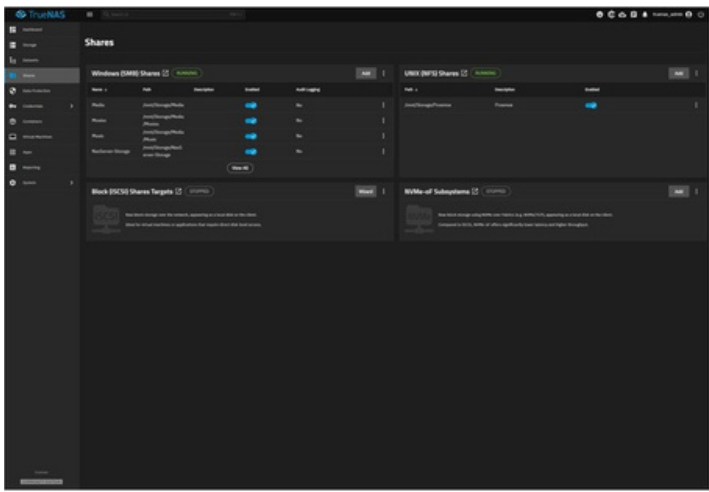
Proxmox host resource utilization (CPU/memory).

STORAGE, BACKUP & ACCESS CONTROL (TRUENAS)

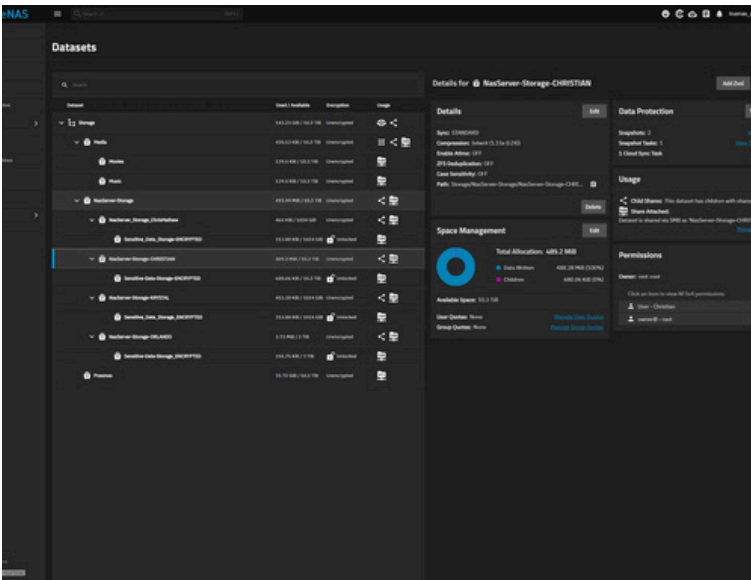
Administer a TrueNAS SCALE server providing network storage for personal files, media, and application data. Organized storage into multiple datasets with scoped permissions, configured SMB and NFS shares with host/network access restrictions, and set up UPS monitoring (NUT) with a properly configured monitor user after diagnosing an initial misconfiguration. Receives and stores the scheduled backups pushed from the Proxmox host over NFS, and runs a small set of self-hosted apps directly on the NAS.



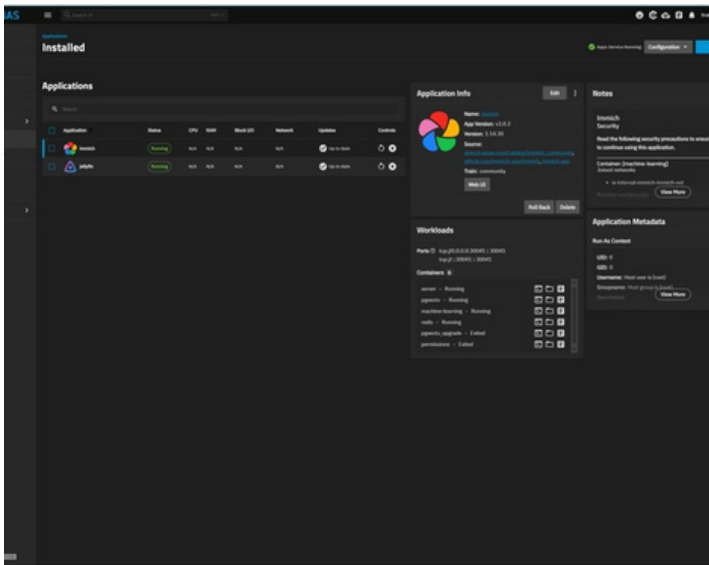
TrueNAS storage dashboard.



TrueNAS SMB/NFS share configuration with access controls.



TruNas expanded datasets and shares



Installed applications on TrueNAS.

SELECTED TROUBLESHOOTING & PROBLEM-SOLVING

A few concrete examples of root-cause diagnosis performed during this project, each representative of the kind of systematic troubleshooting expected in a SOC or help desk role:

- Traced a VPN client showing zero received traffic to a WAN interface setting silently dropping private-sourced packets, rather than assuming a misconfigured tunnel.
- Diagnosed a browser certificate rejection (NET::ERR_CERT_VALIDITY_TOO_LONG) back to a self-signed certificate exceeding the modern 398-day maximum validity policy, not a trust-chain problem, despite the error appearing identical to one.
- Isolated a file-share permission failure to a parent directory's restrictive Linux permissions several levels above the actual shared folder, rather than the share configuration itself, and resolved it with a scoped bind mount instead of loosening broad system directory permissions.
- Identified a severe server performance issue as CPU/I-O contention between a background data pre-generation task and live usage, and resolved it with an automated pause/resume mechanism instead of just reducing overall scope.

SKILLS DEMONSTRATED

- Firewall administration and rule design (least privilege, default-deny)
- VPN deployment and access-tiering (WireGuard)
- IDS/IPS deployment, rule tuning, and alert triage (Snort)
- PKI / certificate lifecycle management (CA creation, issuance, renewal, validity-policy troubleshooting)
- Network storage administration and access control (SMB/NFS, TrueNAS)
- Virtualization and container administration (Proxmox, Docker, LXC)
- Monitoring and automated alerting pipelines (Uptime Kuma, Telegram bot integration)
- Systematic troubleshooting and root-cause analysis across networking, storage, and application layers
- Technical documentation of infrastructure decisions and configurations